

Technical Summary: Exploiting Multicast for Accelerating Collective Communication

Ditto Research Notes

arXiv:2605.22428v1, May 2026

1 Motivation and Problem Setting

This paper targets the latency cost of many-to-many collective communication in large-model training and inference. The motivating observation is simple: collective libraries commonly implement operations such as AllGather and MoE dispatch-style AlltoAll by decomposing them into many one-sided unicast writes. When the same payload must be delivered to multiple receivers, the source injects duplicate copies into the fabric. If those duplicate copies traverse a bottleneck link, they inflate collective completion time and directly hurt training throughput or inference token latency.

The authors argue that this issue is becoming more important for two reasons. First, expert-parallel and tensor-parallel communication domains are growing. In MoE inference and training, EP domains of 64, 128, or larger can make dispatch traffic cross multiple server or scale-up boundaries. Second, serving targets such as sub-5 ms TPOT leave little room for communication variance. When collective communication accounts for 20–30% of runtime, and sometimes more for communication-heavy configurations, even modest collective latency improvements can matter.

The obvious solution is multicast, but the paper carefully distinguishes bandwidth reduction from latency reduction. A multicast scheme is useful only when it removes duplicate traffic from the actual critical bottleneck. In a simple CLOS example, switch-level multicast may reduce source uplink bytes but leave downlink bottlenecks unchanged, producing no end-to-end latency reduction. This point is important: the contribution is not a blanket claim that multicast accelerates collectives, but an attempt to identify where replication should occur so that bottleneck links carry fewer redundant bytes.

2 Why Traditional Multicast Does Not Fit AI Collectives

The paper identifies three practical blockers for conventional network-layer multicast. First, traditional multicast requires pre-created groups and explicit membership management. That is manageable for static groups but problematic for MoE dispatch, where each token may choose a different Top- k expert set. With EP=64 and Top-8 routing, the worst-case number of destination combinations is $\binom{64}{8}$, which exceeds 4.4 billion. Dynamic group join/leave is also too slow for the data plane; tens of microseconds of control-plane overhead would erase the benefit for low-latency collectives.

Second, conventional multicast is not naturally aligned with existing HCCL/NCCL-like programming models. Collective libraries already rely on one-sided read/write semantics, buffer registration, and pairwise unicast transport behavior. Introducing a separate multicast plane, queue-pair attachment, group creation, and multicast-specific reliability semantics would require substantial framework redesign.

Third, hardware multicast often depends on switch or fabric support. The paper’s target is a deployable mechanism on existing Ascend systems without requiring switch modifications. This pushes the design toward an end-host or accelerator-side semantic rather than an in-switch primitive.

3 Core Mechanism: MultiWrite

MultiWrite is a multicast-inspired extension of one-sided remote write. Instead of exposing only `write(src_buf, dst_buf)`, the abstraction accepts a source buffer and a set of destination-memory pairs. Semantically, `MultiWrite`

writes the same payload to all listed destination buffers. Operationally, it carries destination metadata with the packet or transaction, and relay nodes recursively replicate and forward the payload when multiple downstream destinations share a next-hop path.

The design has four key ingredients:

- **Destination metadata instead of multicast groups.** The source embeds destination information in each outgoing transaction. The paper proposes bitmap encoding over NPU ranks, e.g., a 64-bit bitmap for up to 64 ranks. This avoids explicit multicast group creation and makes dynamic MoE destination sets feasible.
- **Reuse of unicast forwarding state.** Each node maps destination ranks to egress ports using existing unicast forwarding tables. For a packet whose bitmap contains multiple destinations, the node computes the union of required egress ports and replicates only as needed.
- **Recursive execution.** If a MultiWrite operation reaches a node with one remaining destination, it degenerates into a normal write. If several destinations remain and share relay structure, the node partitions the destination set by next hop and launches child MultiWrite operations.
- **Integration as a write semantic.** In their UB protocol implementation, MultiWrite is represented as a new transaction opcode. Upper-layer frameworks see an interface similar to normal write, except the destination argument is an array of remote buffers.

The resulting mental model is “multicast without multicast groups.” Replication happens at selected accelerator-side relay nodes rather than switches, and the operation inherits reliability from the underlying unicast transport. The paper explicitly does not guarantee ordering across different destinations, which is acceptable for many collective operators that synchronize at the operation boundary.

4 Beneficial Scenarios

The paper’s most useful systems insight is the condition for benefit: MultiWrite helps when duplicate payload copies would otherwise traverse the bottleneck, and when replication can be moved to a location after that bottleneck.

Full-mesh scale-up topology. The first scenario is an 8-NPU full-mesh system split into two TP=4 communication domains. Traditional AllGather within each TP group uses only intra-domain direct links, leaving cross-domain links idle. A unicast multipath scheme can use those idle links, but it sends redundant copies over the shared cross-domain path. MultiWrite uses the cross-domain path while avoiding redundant copies. This is especially relevant to full-mesh accelerator domains such as Ascend 910B/950-style systems, but the principle generalizes to scale-up islands where some physical links are underutilized by the default collective schedule.

Oversubscribed inter-server MoE dispatch. The second scenario is MoE dispatch across servers. In a server with high intra-server NPU bandwidth but lower inter-server bandwidth, sending multiple copies of the same token to experts on a remote server wastes scarce cross-server capacity. MultiWrite sends one copy across the inter-server bottleneck to a relay NPU, then replicates within the remote server where bandwidth is cheaper. This maps cleanly to EP groups spanning multiple servers and is the more directly relevant case for superpod-scale inference/training.

5 Implementation and Evaluation

The implementation is a software extension on Huawei’s UB/UMDK stack and is evaluated on commercial Ascend 910B4 NPUs. Each server has eight NPUs connected by full-mesh HCCS links at 56 GB/s, with cross-server RoCE connectivity at 200 Gbps per port through a CLOS fabric. The authors implement MultiWrite-based AllGather and AlltoAll/dispatch operators and compare against production HCCL baselines, including some internal baseline optimizations.

For AllGather in the full-mesh TP=4 setup, the paper reports that MultiWrite reduces latency by about 30% versus baseline and 17% versus a unicast multipath implementation at a 16 MB per-rank message size. Across message sizes, the benefit appears only after the message is large enough to amortize splitting, metadata parsing, and relay overhead. The reported crossover is around 2 MB; at 256 KB, MultiWrite is slower than the baseline.

For MoE dispatch-style AlltoAll on 16 NPUs with 64 experts and Top-8 routing, the benefit is batch-size dependent. In decode-like small batches, MultiWrite can be worse or roughly equal because there is too little redundant inter-server traffic to offset relay overhead. In prefill-like larger batches, the paper reports 12% latency reduction at batch

size 1k and 27% at batch size 2k. Their table of cross-server transfer latency shows that removing redundant copies saves increasingly large absolute time as batch size grows.

The main overheads are destination metadata, relay processing, and buffer usage. The metadata overhead is small: 64-bit bitmaps fit into existing immediate fields, while even 1024-rank metadata would consume only 1024 bits. Relay work consumes device-side CPU resources, specifically AICPU on Ascend, but the authors argue that this resource is often underutilized by high-performance data-plane workloads. Relay buffers reuse receive buffers already reserved by collective frameworks, so memory overhead is claimed to be negligible.

6 Limitations and Relevance to AI Datacenter Networks

The design is most convincing as a semantic and scheduling idea rather than as a universal replacement for collective algorithms. It is not useful for all traffic shapes. Small messages, decode-path MoE, or topologies where the bottleneck is not on the shared duplicate path may see no gain or regression. The relay node can also become a new hotspot: MultiWrite moves replication work downstream, which reduces upstream load but may create burstier fanout traffic and device-side scheduling pressure. A production deployment would need careful interaction with congestion control, credit management, and collective scheduling.

The evaluation is communication-only, not end-to-end model throughput. That is methodologically cleaner than inflating model-level gains with contrived communication fractions, but it means the actual training or serving improvement depends on the workload’s communication share. For example, a 30% collective improvement on a phase that consumes 20% of runtime implies a single-digit end-to-end improvement before secondary effects.

For scale-up and superpod design, the paper is relevant because it frames multicast as a *memory/transaction semantic* rather than a switch feature. That creates an interesting design space for SmartNICs, accelerator NICs, and collective engines: expose multi-destination writes, encode dynamic destination sets, select relay points based on bottleneck structure, and let the transport preserve reliability. The key open question is how to make this portable beyond Ascend/UB/HCCCL, especially in GPU clusters using NCCL, NVLink/NVSwitch, IB/RoCE, and NIC-level congestion-control mechanisms.

7 Bottom Line

MultiWrite is a practical multicast-like abstraction for AI collectives that avoids traditional multicast group management. Its value comes from removing redundant payload copies from bottleneck links, especially in full-mesh scale-up domains and oversubscribed inter-server MoE dispatch. The idea is strong, but deployment success depends on message size, bottleneck placement, relay overhead, and integration with collective libraries and transport control loops.